GRADE 11 TECHPRO ACTIVITY SHEET

CSS FLEXBOX: CONTAINERS, ALIGNMENT, ITEMS, AND RESPONSIVE LAYOUTS

ICT / COMPUTER PROGRAMMING | GRADE 11 | VISUAL STUDIO CODE

Name: Ramo, Jeric Y. Section: 11 - Bell Date: 11 / 9 / 26

LEARNING OBJECTIVES

Explain the purpose of CSS Flexbox and identify a flex container and its flex items.

Apply flex-direction, flex-wrap, and gap to arrange items.

Use justify-content and align-items to control alignment.

Use flex-item properties: order, flex-grow, flex-shrink, flex-basis, flex, and align-self.

Build a responsive layout using a media query.

QUICK REVIEW

Flexbox is a one-dimensional layout system for arranging items in a row or column.

display: flex turns an element into a flex container. Its direct children become flex items.

flex-direction controls row or column direction. flex-wrap allows items to move to another line. gap adds space between items.

justify-content aligns items along the main axis; align-items aligns items along the cross axis.

align-content controls spacing between multiple flex lines when wrapping occurs.

order changes visual order. flex-grow controls growth; flex-shrink controls shrinking; flex-basis sets an initial size; flex is shorthand; align-self overrides an individual item's cross-axis alignment.

A media query applies CSS rules at selected screen widths to adapt a layout.

WHAT YOU NEED

Computer with Visual Studio Code and a web browser.

Create a folder named CSS_Flexbox_Activity and a file named index.html.

Copy the starter code below, save it, and open the file in a browser.

STARTER CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>TechPro Flexbox Page</title>
<style>
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; line-height: 1.5; }
header { background: #7b1530; color: white; padding: 20px; text-align:
center; }
nav { display: flex; justify-content: space-between; align-items: center;
gap: 12px; background: #f3e5e9; padding: 12px 20px; }
nav a { color: #7b1530; text-decoration: none; font-weight: bold; }
main { max-width: 1000px; margin: auto; padding: 20px; }
.flex-container { display: flex; flex-direction: row; flex-wrap: wrap;
justify-content: space-evenly; align-items: stretch; gap: 15px; background:
#eee; padding: 15px; min-height: 180px; }
.card { flex: 1 1 220px; background: white; border: 2px solid #7b1530;
padding: 18px; text-align: center; }
```

```
.featured { order: 2; flex-grow: 2; }
.special { align-self: center; }
footer { background: #222; color: white; padding: 15px; text-align: center; }
@media (max-width: 600px) {
  nav { flex-direction: column; align-items: stretch; }
  .flex-container { flex-direction: column; }
  .card { flex: 1 1 auto; }
}
</style>
</head>
<body>
<header><h1>TechPro Learning Hub</h1><p>Learning CSS Flexbox</p></header>
<nav><a href="#">Home</a><a href="#">Lessons</a><a href="#">Activities</a><a
href="#">Contact</a></nav>
<main>
<h2>Our Learning Cards</h2>
<p>Resize the browser window to test the responsive layout.</p>
<section class="flex-container">
<article class="card">HTML Basics</article>
<article class="card featured">CSS Styling</article>
<article class="card special">JavaScript</article>
</section>
</main>
<footer>Grade 11 TechPro | CSS Flexbox Activity</footer>
</body>
</html>
```

ACTIVITY 1 — CREATE A FLEX CONTAINER

Find .flex-container in the CSS. Confirm that it has display: flex.

Change flex-direction from row to column. Save and refresh the browser. Change it back to row.

Change flex-wrap from wrap to nowrap, then resize the browser. Restore flex-wrap: wrap.

Change gap from 15px to 30px.

Record: What happens to child elements when display: flex is applied? What does flex-wrap do?

ACTIVITY 2 — PRACTICE JUSTIFY-CONTENT

Find justify-content: space-evenly in .flex-container.

Test these values one at a time: flex-start, center, flex-end, space-between, space-around, and space-evenly.

Save and refresh after each change. Observe the horizontal spacing between the cards.

Restore justify-content: space-evenly.

Record: Which value places equal space around items? Which value puts items at the end?

ACTIVITY 3 — PRACTICE ALIGN-ITEMS

Find align-items: stretch. Change it to center.

Set min-height of .flex-container to 220px so vertical alignment is easier to see.

Test flex-start, flex-end, center, and stretch. Restore align-items: stretch.

Record: Which property aligns items on the cross axis? How is it different from justify-content?

ACTIVITY 4 — EXPLORE FLEX ITEMS

Find .featured. Change order: 2 to order: -1. Observe its new visual position.

Change flex-grow: 2 to flex-grow: 3. Compare the featured card's width with the other cards.

Add flex-shrink: 1 and flex-basis: 250px to .featured. Resize the browser.

Find .special. Add align-self: flex-end, then try flex-start. Restore align-self: center.

Record: Which property changes visual order? Which property overrides align-items for one item?

ACTIVITY 5 — MAKE THE PAGE RESPONSIVE

Find the @media (max-width: 600px) rule at the bottom of the CSS.

Change 600px to 700px. Save and resize the browser around that width.

Inside the media query, change .flex-container to flex-direction: column.

Observe how cards stack vertically on a narrow screen. Test wide and narrow views.

Record: What is a media query breakpoint? Why test a page at different widths?

MINI-PROJECT — TECHPRO COURSE CARD LAYOUT

Customize the starter page into a TechPro course page with at least four course cards.

☐ Use display: flex on the card container.

☐ Use flex-wrap: wrap and a gap of at least 12px.

☐ Use justify-content and align-items to create a neat arrangement.

☐ Give each card a flexible starting width using flex or flex-basis.

☐ Use order or align-self on at least one card.

☐ Add a media query so cards stack vertically on small screens.

☐ Keep the maroon, white, and light-gray color theme.

☐ Check that there is no unwanted horizontal scrolling on a narrow screen.

CHECKLIST AND REVIEW QUESTIONS

☐ I created and saved index.html.

☐ I tested flex-direction, flex-wrap, and gap.

☐ I tested at least four justify-content values and three align-items values.

☐ I used at least two flex-item properties.

☐ I tested the page on wide and narrow browser widths.

1. What CSS declaration creates a flex container?
   display: flex;
2. What is the difference between justify-content and align-items?
   justify-content aligns items along the main axis, while align-items aligns items along the cross axis
3. What does flex-wrap: wrap do?
   allows flex items to break onto multiple lines when they run out of space inside their container
4. What is the purpose of flex-grow?
   to define how much a flex item can expand to fill any remaining positive free space inside its parent flex container
5. What does align-self control?
   the alignment of an individual flex or grid item along the container's cross-axis
6. Why is a media query useful in responsive Flexbox design?
   allows you to changethe structural layout of containers within specific device viewports

REFERENCES

W3Schools — CSS Flexbox: https://www.w3schools.com/css/css3_flexbox.asp

W3Schools — Flexbox Container: https://www.w3schools.com/css/css3_flexbox_container.asp

W3Schools — justify-content: https://www.w3schools.com/css/css3_flexbox_container_justify.asp

W3Schools — align-items and align-content: https://www.w3schools.com/css/css3_flexbox_container_align.asp

W3Schools — Flexbox Items: https://www.w3schools.com/css/css3_flexbox_items.asp

W3Schools — Responsive Flexbox: https://www.w3schools.com/css/css3_flexbox_responsive.asp